M6A1: Problem Set — Kossiakoff Chapter 13, Problem 13.2

EN.655.662 Introduction to Healthcare Systems Engineering | [Your Name] | [Date]

System of interest: the remote cardiac care system — an implanted or worn cardiac device, a patient-home desk transceiver, a cellular transport link, a clinical computer stack, and a clinician-facing clinical station.

Why Advanced Development Exists

Advanced development sits between concept definition and engineering design. Concept definition ends with a chosen system concept; engineering design begins the expensive and largely irreversible commitment to a specific physical realization. Advanced development is the interval in which the organization is still free to change its mind cheaply. Its product is therefore not a design — it is **knowledge**, specifically the retirement of the uncertainties that would otherwise be carried into a phase where correcting them is costly. That is the whole reason the phase is called risk reduction.

Activities of the Phase, and the Risk Each Retires

Requirements analysis and flow-down. Operational requirements from concept definition are decomposed into system and subsystem requirements that are measurable, verifiable, and traced to the functions they serve. This retires *requirements risk*: the failure mode in which a requirement proves ambiguous or unverifiable only after a design has been built to satisfy it. In the remote cardiac system, an operational requirement such as "minimum communications range of 40 ft" flows down into system requirements such as a power-up time no greater than 5 s, an activation force no greater than 1 N, and a power confirmation cue specified as a 55 dB, 2 Hz, 200 ms chirp. Each of those is testable; the parent requirement by itself is not.

Technology identification and maturation. Each element is assessed for maturity, conventionally by technology readiness level, and immature elements are matured deliberately rather than by hope. This retires *technology risk*. The remote cardiac system is not uniformly mature: the clinical computer stack and the cellular transport are effectively off-the-shelf at TRL-9, the desk transceiver and clinical station sit near TRL-7, and end-to-end operation with a live cardiac device and patient is least mature at roughly TRL-6. The TRL assessment is what tells the programme where to spend its prototyping budget, and here it points at the integration rather than at any individual box.

Prototyping. Prototypes are built to answer questions, not to be the product. They may be explorative (developed with users to surface requirements), experimental (built to determine whether the planned system will be adequate), or evolutionary (successively refined into the delivered system), and they span paper sketches, simulations, and first-article builds. Prototyping retires *feasibility and usability risk* at a fraction of the cost of discovering the same facts during engineering design.

Modeling, simulation and analysis. Where a prototype is expensive or premature, an explicit model bounds system behaviour analytically. This retires *performance and integration risk* before hardware exists and supports early, iterative verification and validation instead of deferring all of it to a downstream test phase.

Trade studies. Alternatives are identified and compared against measures of feasibility, performance, cost, schedule and risk, with estimates drawn from modeling, prototyping or surrogates. This retires *selection risk* — committing to an architecture that turns out to be a dead end — and leaves a documented rationale that survives staff turnover.

Interface definition and developmental test planning. Interfaces between elements are specified, and the test and evaluation master plan, test equipment and test software are developed. This retires

integration risk, the dominant risk in a distributed system of this kind, and *verification risk*, the late discovery that a requirement cannot be tested as written.

Production of the validated system specification. The phase exits by baselining a system specification that has been shown to be achievable, so engineering design proceeds against a stable target. Without that exit artifact the next phase inherits an unvalidated baseline, which is precisely the condition in which technical uncertainty is carried forward into full-scale development.

Why the Phase Is Called "Risk Reduction"

Two arguments converge. The first is economic: by the close of conceptual and advanced development the great majority of total product cost is already committed even though very little has been spent, so a change made here is inexpensive and the identical change made during engineering design or production is not. The second is epistemic: every other phase produces an artifact — a design, a build, a fielded system — whereas advanced development produces evidence. Its deliverables are a matured technology base, a validated specification, and a set of demonstrated feasibilities. A programme that skips the phase has not avoided the uncertainty; it has only relocated the discovery of that uncertainty to a phase where it is expensive to act on.

Example: Risk Reduction in the Remote Cardiac Care System

The dominant risk in this system is not any single component but the end-to-end chain, where the least mature element sits. A disciplined advanced development programme therefore builds the system up in an order that introduces exactly one new source of uncertainty at a time: confirm and mature the desk transceiver in isolation; isolate and test the clinical computer stack; isolate and test the clinical station; combine the station and the stack; add the cellular link for a partial end-to-end test; run end-to-end without the cardiac device; run end-to-end with the cardiac device; and only then run a complete end-to-end test with a patient. The logic is that when a test fails, the cause is unambiguous, because only one thing changed. Integrating two unknowns simultaneously destroys that property, and is how integration schedules collapse.

A concrete instance makes the payoff visible. Cellular transport between the patient’s home and the clinical stack can be modelled for average latency and throughput, and the model will show the "data rate 2× that of the cardiac device" requirement comfortably satisfied. What the model will not reveal is intermittent dropout inside a particular patient’s home — a basement bedroom, a steel-framed building, a carrier with poor indoor coverage. That is discoverable only by a field prototype of the transceiver and link, which is exactly what the first and fifth steps of the build-up exist to do. If the dropout is found during advanced development, the response is an architectural change that is still cheap: add local store-and-forward buffering to the desk transceiver with a defined retention window and reconnection behaviour, and flow a new requirement down accordingly. If the same dropout is found after engineering design — or worse, in the field — the response is a hardware respin of a device already in production, a regulatory change assessment, and a clinical safety case built around the possibility that an arrhythmia event was never transmitted. The engineering work is substantially the same; the cost and the patient consequence differ by orders of magnitude.

One discipline matters throughout. Prototypes in this phase should stay at the fidelity the question requires. Going high-fidelity early is a recognised failure mode: it consumes schedule, it can lead a sponsor to believe the system is nearly finished, and a polished prototype can quietly constrain the eventual design. The prototype that answers the coverage question needs a radio, an antenna and a data logger. It does not need an enclosure.